

DB Field Gap Analysis — proposals & orders

Analysis of which DB columns the proposal generator currently fills, which it could fill from existing data with small wiring changes, and which require new inputs (form fields, lookups, or downstream automation).

Sources of truth inspected:

- `contexts/ProposalContext.tsx` — in-memory shape of the proposal
- `app/page.tsx` — form inputs
- `app/preview/page.tsx` — preview/edit and POST trigger
- `app/api/generate-proposal/route.ts` — server-side payload build + `save_proposal_detail`
- `lib/utils.js` — `save_proposal_detail`, `getClientDetails`, `uploadProposalFiles`

1. What the app currently collects (proposal payload)

Group	Fields
<code>clientInfo</code>	<code>clientNumber</code> , <code>companyName</code> , <code>street</code> , <code>postalCode</code> , <code>city</code> , <code>country</code>
<code>projectInfo</code>	<code>projectNumber</code> , <code>projectName</code> , <code>projectType</code> , <code>customProjectType</code> , <code>deliveryTime</code> (text, e.g. "4–6 Werktag"), <code>deliveryDaysMin</code> , <code>deliveryDaysMax</code> , <code>offerValidUntil</code> , <code>date</code> , <code>MM</code> , <code>DD</code> , <code>year</code>
<code>services[]</code>	<code>id</code> , <code>name</code> , <code>quantity</code> , <code>unitPrice</code> , <code>totalPrice</code> , <code>customPrice?</code> , <code>buildingType?</code> , <code>apartmentSize?</code> , <code>projectType?</code> , <code>areaSize?</code> , <code>customDescription</code>
<code>pricing</code>	<code>subtotalNet</code> , <code>discountAmount</code> , <code>totalNetPrice</code> , <code>totalVat</code> , <code>totalGrossPrice</code> , <code>discount {type, value, amount, description}</code>
<code>signature</code>	<code>signatureName</code>
<code>images[]</code>	<code>title</code> , <code>description</code> , <code>fileName</code> , <code>fileSize</code> , <code>fileType</code> , <code>imageData</code>
<code>terms</code>	passed through to template but currently {}
Server-derived	<code>offerNumber</code> (generated), <code>storageFolderPath</code> , <code>docxUrl</code> , <code>pdfUrl</code> , <code>docxFilename</code> , <code>pdfFilename</code> , <code>imagesIncluded</code> , <code>dbClientData.client_id</code> (from Supabase lookup)

2. proposals table — column-by-column status

Legend:

- OK** — written today
- WIRING** — data exists in the payload but isn't saved (or is buried inside `pricing/document_url` jsonb)
- NEW FIELD** — needs a new form input, lookup, or external source

- **AUTO** — should be set automatically (defaults, triggers, or server logic)

Column	Status	Notes / Source
<code>id</code>	OK	serial PK
<code>client_id</code>	OK	<code>dbClientData.client_id</code> from <code>getClientDetails</code>
<code>company_name</code>	OK	from form / DB fallback
<code>street_no</code>	OK	<code>clientInfo.street</code>
<code>city</code>	OK	form
<code>country</code>	OK	form, defaults to <code>'Deutschland'</code>
<code>postal_code</code>	OK	form
<code>project_number</code>	OK	form
<code>project_name</code>	OK	form
<code>project_type</code>	OK	form
<code>offer_number</code>	OK	server-generated
<code>delivery_time_min</code>	OK (buggy)	currently parsed from <code>projectInfo.deliveryDays</code> which no longer exists — context exposes <code>deliveryDaysMin/Max</code> . Fix: read <code>projectInfo.deliveryDaysMin</code> directly.
<code>delivery_time_max</code>	OK (buggy)	same bug as above
<code>services</code>	OK	jsonb
<code>pricing</code>	OK	jsonb
<code>discount_type</code>	OK	from <code>pricing.discount.type</code>
<code>discount_value</code>	OK (buggy)	currently reads <code>pricing.discount.amount</code> (a money string) instead of <code>pricing.discount.value</code> (the % or fixed amount). Two columns are conflated — see <code>discount_amount</code> row below.
<code>currency</code>	OK	hard-coded <code>'EUR'</code>
<code>total_price</code>	OK (buggy)	code does <code>pricing.totalGrossPrice?.replace(...)</code> but <code>totalGrossPrice</code> is a number , not a string — this will throw at runtime. Fix: assign the number directly.
<code>image_urls</code>	OK (partial)	currently saves <code>[{title, description}]</code> only; no actual URLs. Once images are uploaded to storage, add their URLs here.
<code>document_url</code>	OK	jsonb <code>{docx, pdf, folder}</code>

Column	Status	Notes / Source
<code>created_at</code>	AUTO	DB default <code>now()</code>
<code>updated_at</code>	AUTO	DB default <code>now()</code> (consider a trigger to bump on UPDATE)
<code>proposal_date</code>	WIRING	<code>projectInfo.date</code> exists — map to <code>proposal_date</code> (cast to <code>date</code>).
<code>offer_valid_until</code>	WIRING	<code>projectInfo.offerValidUntil</code> exists in payload but isn't persisted.
<code>delivery_time_text</code>	WIRING	<code>projectInfo.deliveryTime</code> ("4–6 Werkstage") is in the payload but not saved.
<code>custom_project_type</code>	WIRING	<code>projectInfo.customProjectType</code> already collected.
<code>signature_name</code>	WIRING	<code>signature.signatureName</code> collected, never saved.
<code>client_input_identifrier</code>	WIRING	the raw value the user typed in the client lookup (<code>clientInfo.clientNumber</code> — may be a number, email, or company name per <code>getClientDetails</code>). Useful for audit/debug.
<code>subtotal_net</code>	WIRING	<code>pricing.subtotalNet</code> — promote to a top-level column for indexing/reporting.
<code>total_net_price</code>	WIRING	<code>pricing.totalNetPrice</code>
<code>total_vat</code>	WIRING	<code>pricing.totalVat</code>
<code>vat_rate</code>	WIRING	hard-coded <code>0.19</code> in <code>recalculatePricing</code> ; persist as <code>19.00</code> (column default already <code>19.00</code> — fine to leave unless rate becomes configurable).
<code>discount_description</code>	WIRING	<code>pricing.discount.description</code> available; not saved.
<code>discount_amount</code>	WIRING	<code>pricing.discountAmount</code> (the money computed value) — distinct from <code>discount_value</code> (the input %/fixed amount). The current code stores the money string in <code>discount_value</code> and never writes <code>discount_amount</code> . Fix: <code>discount_value = discount.value</code> (raw <code>10 / 50 /</code> etc.), <code>discount_amount = pricing.discountAmount</code> .
<code>docx_url</code>	WIRING	<code>fileUrls.docxUrl</code> exists — promote out of <code>document_url</code> jsonb to its own column.
<code>pdf_url</code>	WIRING	same; <code>fileUrls.pdfUrl</code> .
<code>storage_folder_path</code>	WIRING	<code>fileUrls.storagePath</code> exists.
<code>docx_filename</code>	WIRING	constructed locally (<code>filename</code>); not saved.
<code>pdf_filename</code>	WIRING	constructed locally (<code>pdfFilename</code>); not saved.

Column	Status	Notes / Source
<code>images_count</code>	WIRING	<code>images.length</code> available.
<code>terms</code>	WIRING	passed through but currently <code>{}</code> ; persist whatever the template renders so later edits aren't lost.
<code>raw_payload</code>	WIRING	save the whole request body for audit/replay.
<code>payload_version</code>	WIRING	introduce a constant like <code>'1.0.0'</code> in the generator and bump on schema changes.
<code>path_to_files</code>	WIRING	same as <code>storage_folder_path</code> or a local mirror — clarify intent.
<code>proposal_status</code>	AUTO	default to <code>'draft'</code> . Set <code>'sent'</code> when n8n webhook returns 200, <code>'viewed'</code> from a recipient tracking pixel, <code>'accepted'/'rejected'</code> from a customer reply flow.
<code>project_id</code>	NEW FIELD	text FK into <code>projects(project_id)</code> . The form has <code>projectNumber</code> (a free-text human reference) but not the canonical <code>project_id</code> . Either (a) generate it server-side and upsert a <code>projects</code> row, or (b) require the automation to look up / create the project first and pass the id in.
<code>project_manager</code>	NEW FIELD	not collected. Add a PM selector to the form, or assign from rules (round-robin, by region, by service type).
<code>pm_type</code>	NEW FIELD	enum <code>pm_type</code> . Same source as <code>project_manager</code> — likely a derived/joined value once PM is set. Trigger could fill it.
<code>sales_person</code>	NEW FIELD	not collected. Could default to the logged-in user once auth is wired (note: there's a <code>users</code> table in <code>lib/supabase.ts</code>).
<code>email_id</code>	NEW FIELD	FK to <code>emails(email_id)</code> . When the proposal is triggered from an inbound email automation, pass the originating email id. The lookup in <code>getClientDetails</code> already touches the <code>emails</code> table — surface that id.
<code>person_id</code>	NEW FIELD	FK to <code>persons(person_id)</code> . Need a person-level lookup (the current lookup only resolves to a company). Add <code>contact_email/contact_name</code> inputs and resolve to a <code>person_id</code> .
<code>company_email</code>	WIRING-ish	not on the form, but the lookup already queries by email. If the user types an email into the client lookup, save it here. Otherwise add a "primary contact email" form field.
<code>client_contact_name</code>	NEW FIELD	add a contact-name field to the client section (the salutation in the letter currently hard-codes nothing — also

Column	Status	Notes / Source
		useful for Sehr geehrte/r ...).
construction_type	NEW FIELD	enum construction_type_values . Add a dropdown (likely Neubau / Bestand / ...).
property_type	NEW FIELD	enum property_type_values . Distinct from project_type ("EFH", "MFH-3-5", ...) which is the building form factor — clarify which is which with the data team.
questionnaire_received	NEW FIELD	enum yes/no. Add a checkbox.
deposit	NEW FIELD	enum yes/no. Add a checkbox.
initial_project_status	NEW FIELD	enum project_status_values . Default at creation (e.g. 'lead' / 'offer_sent').

Quick fixes (bugs in the current INSERT)

These don't require schema changes — just edits to **app/api/generate-proposal/route.ts**:

1. **delivery_time_min/max** — **projectInfo.deliveryDays** doesn't exist on the context; use **deliveryDaysMin / deliveryDaysMax**.
2. **total_price** — **totalGrossPrice** is a number; calling **.replace()** on it throws. Assign directly.
3. **discount_value vs discount_amount** — store **discount.value** (raw input) in **discount_value** and **pricing.discountAmount** (computed money) in **discount_amount**.
4. **image_urls** — currently stores **{title, description}** placeholders; either rename the column intent or upload images to storage and store real URLs.

3. **orders** table — gap to "auto-create from proposal"

The **orders** table represents a **single product/service line per row** (it has **product**, **quantity**, **unit_price**, **supplier**, individual delivery dates, etc.). The proposal carries an **array of services**. So "creating orders from a proposal" means **fanning out one order row per service** (or per service quantity, depending on business rules).

Several triggers already auto-fill fields from **project_id**:

- **trg_autofill_person_id**, **trg_orders_set_pm_from_project**, **trigger_set_pm_type**, **trg_orders_set_financial_fields**, **trg_orders_set_order_id**, **trg_orders_set_product_from_name**.

So if you provide **project_id + product_name + quantity + unit_price**, **a lot of the table fills itself**.

Column	Status	Notes
id	AUTO	identity

Column	Status	Notes
<code>project_id</code>	NEW FIELD (critical)	FK to <code>projects(project_id)</code> . Blocker — see <code>proposals.project_id</code> above.
<code>order_id</code>	AUTO	filled by <code>trg_orders_set_order_id</code>
<code>order_number</code>	NEW FIELD	distinct from <code>order_id</code> ? Confirm with data team — may be a customer-facing number.
<code>net_sum</code>	AUTO	from <code>trg_orders_set_financial_fields</code> (likely <code>quantity * unit_price</code>)
<code>gross_sum</code>	AUTO	same trigger
<code>db_1, profit_margin, roi</code>	AUTO/ NEW	from financial fields trigger if it computes from <code>cost</code> ; otherwise needs <code>cost</code> input
<code>ap_epcs_invoicing</code>	NEW FIELD	invoicing routing label — not in proposal scope
<code>comments</code>	optional	could carry the service's <code>customDescription</code> summary
<code>order_type</code>	NEW FIELD	enum <code>order_type</code> . Probably constant per proposal-origin (e.g. <code>'sale'</code>)
<code>product_type</code>	NEW FIELD	enum <code>product_type</code> . Derivable from the service id (e.g. <code>interior</code> → <code>'visualization'</code>) once a mapping table exists
<code>sale_type</code>	NEW FIELD	enum <code>sale_type</code> . Likely constant for proposal-origin orders
<code>quantity</code>	OK	<code>service.quantity</code>
<code>product</code>	OK-ish	free-text — <code>service.name</code> works; but <code>product_name</code> below is the FK
<code>unit_price</code>	OK	<code>service.unitPrice</code> (parse to numeric)
<code>supplier</code>	NEW FIELD	unknown at proposal time — assigned during fulfilment
<code>cost</code>	NEW FIELD	supplier cost — assigned during fulfilment
<code>date_information_complete</code>	NEW FIELD	post-proposal, when full project brief is in
<code>due_delivery_date</code>	WIRING	derivable: <code>proposal_date + delivery_time_max</code> business days. Worth computing on insert.
<code>delivery_1_date...</code> <code>delivery_4_date</code>	NEW FIELD	tranche delivery — fulfilment data
<code>delay_first_delivery,</code> <code>delay_first_revision,</code>	AUTO	default 0

Column	Status	Notes
<code>delay_second_revision</code>		
<code>product_name</code>	NEW MAPPING	FK to <code>product_codes(name)</code> . Needs a service-id → product_codes.name mapping table. This is the linchpin that lets <code>trg_orders_set_product_from_name</code> populate <code>product</code> . Build this mapping in <code>lib/services.ts</code> (add a <code>productCode</code> field per service) or as a DB lookup.
<code>PM</code>	AUTO	from project trigger
<code>pm_type</code>	AUTO	from project trigger
<code>person_id</code>	AUTO	from project trigger
<code>company_name</code>	AUTO	from project trigger
<code>client_rating</code>	NEW FIELD	enum — out of scope for proposal generation
<code>questionnaire_received</code>	NEW FIELD	same source as the proposal copy
<code>deposit</code>	NEW FIELD	same source as the proposal copy
<code>supplier_payment</code>	NEW FIELD	fulfilment-time boolean
<code>project_completion_date</code>	NEW FIELD	fulfilment-time
<code>click_up_task_link</code>	NEW FIELD	fulfilment-time
<code>discount</code>	WIRING	derivable per line from the proposal-level discount, only if discounts are applied per-line. Today the discount is global. Decide: (a) split the global discount proportionally across orders, or (b) keep <code>orders.discount = null</code> and only show discount on the proposal.
<code>created_at, updated_at</code>	AUTO	defaults / <code>trg_set_updated_at</code>

What's needed to **automate orders from proposals**

In rough priority order:

1. **Resolve `project_id`** — either generate one in the generate-proposal route (insert into `projects` first, get back the id) or require the upstream automation to pass it in.
2. **Service → `product_codes.name` mapping** — without this the FK fails. Add `productCode: string` per entry in `lib/services.ts` (or a separate `lib/service_product_map.ts`), and confirm every code exists in `product_codes`.
3. **Decide order granularity** — one order per service line, or one order per (service, quantity)? Affects how you split `quantity` and `unit_price`.
4. **Decide who creates orders** — server-side at proposal generation (simplest, but means orders are "open" before the customer signs), or only after `proposal_status = 'accepted'`

(recommended).

5. **Handle the global discount** — pick a strategy (proportional split / keep null / discount the largest line only).

4. Recommended minimum schema-aware payload (v1)

The cheapest thing to ship first: **change `app/api/generate-proposal/route.ts` so `proposalDbData` populates everything that's already in the payload**, and stash the rest in `raw_payload`.

```
const proposalDbData = {
  // identity
  client_id: dbClientData ? parseInt(dbClientData.client_id) || null :
  null,
  client_input_identfier: clientInfo.clientNumber || null,
  company_name: clientInfo.companyName,
  // address
  street_no: clientInfo.street,
  city: clientInfo.city,
  country: clientInfo.country || 'Deutschland',
  postal_code: clientInfo.postalCode,
  // project
  project_number: projectInfo.projectNumber || null,
  project_name: projectInfo.projectName || null,
  project_type: projectInfo.projectType || null,
  custom_project_type: projectInfo.customProjectType || null,
  // offer
  offer_number: generatedOfferNumber,
  proposal_date: projectInfo.date || null,           // ensure ISO yyyy-mm-dd
  offer_valid_until: projectInfo.offerValidUntil || null,
  delivery_time_min: projectInfo.deliveryDaysMin ?? null,
  delivery_time_max: projectInfo.deliveryDaysMax ?? null,
  delivery_time_text: projectInfo.deliveryTime || null,
  // money (top-level numerics, no string parsing)
  currency: 'EUR',
  vat_rate: 19.00,
  subtotal_net: pricing.subtotalNet,
  total_net_price: pricing.totalNetPrice,
  total_vat: pricing.totalVat,
  total_price: pricing.totalGrossPrice,
  discount_type: pricing.discount?.type || null,
  discount_value: pricing.discount?.value ?? null,    // raw input (10,
  50, ...)
  discount_amount: pricing.discountAmount ?? null,    // computed money
  discount_description: pricing.discount?.description || null,
  // payload jsonb
  services,
  pricing,
  terms: terms || {},
  // files
  docx_url: fileUrls?.docxUrl || null,
  pdf_url: fileUrls?.pdfUrl || null,
```



```

storage_folder_path: fileUrls?.storagePath || null,
docx_filename: filename,
pdf_filename: pdfFilename,
images_count: images.length,
image_urls: imageMetadata?.map((img) => ({ title: img.title, description:
img.description })) || [],
document_url: fileUrls ? { docx: fileUrls.docxUrl, pdf: fileUrls.pdfUrl,
folder: fileUrls.storagePath } : null,
// people / signature
signature_name: signature?.signatureName || 'Christopher Helm',
// status + audit
proposal_status: 'draft',
raw_payload: data,
payload_version: '1.0.0',
};

```

Everything still null after this change (`project_id`, `project_manager`, `pm_type`, `sales_person`, `email_id`, `person_id`, `company_email`, `client_contact_name`, `construction_type`, `property_type`, `questionnaire_received`, `deposit`, `initial_project_status`, `path_to_files`) is the **real list of new inputs** needed before the table is fully driven by the generator.

5. Suggested new form additions (v2)

Grouped to minimise UX churn:

Client section

- `client_contact_name` — single line
- `company_email` — single line (and use it to drive the `getClientDetails` lookup so `client_input_identifier` and `company_email` are coherent)

Project section

- `construction_type` — dropdown (enum)
- `property_type` — dropdown (enum) (*confirm with data team how this differs from existing `projectType`*)
- `project_manager` + (derived) `pm_type` — dropdown of users
- `sales_person` — dropdown of users (default to logged-in)
- `questionnaire_received` — yes/no toggle
- `deposit` — yes/no toggle
- `initial_project_status` — dropdown (enum), default sensible value

Hidden / automation-only

- `email_id` — passed in via query string when launched from an n8n email automation
- `person_id` — resolved server-side once `client_contact_name` + `company_email` are present
- `project_id` — generated server-side (`projects` insert first)

6. Open questions for the data team

1. Is `project_type` ("EFH", "MFH-3-5", ...) the same dimension as `property_type` (`property_type_values` enum), or are these different things? The current form treats it like building form factor.
2. Should orders be created at proposal-time (`status='draft'`) or only on `proposal_status='accepted'`?
3. For a multi-service proposal, do we want **one order per service** or **one order with all services rolled up**? The schema's per-row product/quantity/price strongly implies the former.
4. Is `order_number` user-facing and distinct from the trigger-assigned `order_id`?
5. What is the canonical mapping from `services.ts` ids (e.g. `interior`, `exterior-bird`, `3d-floorplan`) to `product_codes.name`?
6. Is `path_to_files` meant to be the Supabase Storage path (`storage_folder_path`), a network drive path, or something else?